

The Heterogeneous Fleet Vehicle Routing Problem with Draft Limits

Mathematical Optimization - A.A. 2024-2025

Elma Huseinovic, Elisabetta Chisso

Index

- **Context and Problem description**
- **Formulation** (Objective Function, constraints and valid inequalities)
- **Matheuristics** (LNS and ILS)
- **Dataset**
- **Results**
- **Comparison** (Model vs Matheuristics)
- **Conclusions**

Problem Description

Context: In modern maritime transport, naval gigantism has led to increasingly large vessels with greater drafts

Operational issue: Ports have draft limits-ships that are too heavily loaded cannot enter them, which affects the sequence of port visits.

Novelty:

- Incorporates load-dependent draft limits into routing decisions.
- Considers a heterogeneous fleet (ships with different capacities, costs, and drafts).
- Integrates fleet sizing and routing under draft constraints.

Objective: Minimize the total network cost (port access + sailing) by deciding how many and which ships (of different sizes) to use, and in what sequence to visit the ports.

Formulation

SETS

- $I = [1, I_{max}] \rightarrow$ set of ports
- $I0 = [0, I_{max}] \rightarrow$ set of ports including the depot
- $S = [1, S_{max}] \rightarrow$ set of ships

PARAMETERS

- Q_s → Capacity (tons) of the ship s
- c_s → Hourly sailing cost for ship s (€/h)
- q_i → Demand (tons) of the port i
- $coords_i$ → Coordinates of the port i
- L_{is} → Maximum loading for ship s to access port i (tons)
- r_{is} → Access cost for ship s entering port i (€)
- $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ → Euclidean distance between ports i and j
- $t_{ij} = \frac{d_{ij}}{\omega}$ → Sailing time between ports i and j (h)

VARIABLES

- l_{is} → Loading of ship s entering port i
- $u_i \in N^+ \quad \forall i \in I$ → Position of port i in the sequence of visited ports
- $p_s = \sum_{\{i \in I\}} q_i Y_{is} \quad \forall s \in S$ → Total load for ship s

DECISION VARIABLES

- $X_{ijs} \in \{0, 1\} \quad \forall i \in I_0 \quad \forall j \in I_0 \quad \forall s \in S$ Takes value 1 if the arc (i, j) is traversed by ship s
- $Y_{is} \in \{0, 1\} \quad \forall i \in I \quad \forall s \in S$ Takes value 1 if port i is served by ship s

OBJECTIVE FUNCTION

The goal is to minimize the total network cost

$$\min(\sum_{i \in I_0} \sum_{j \in I_0} \sum_{s \in S} c_s t_{ij} X_{ijs} + \sum_{i \in I} \sum_{s \in S} r_{is} Y_{is})$$

The first term represents the sailing cost, and the second represents the sum of the fixed costs to access ports

CONSTRAINTS

1) Each port is assigned to a ship

$$\sum_{s \in S} Y_{is} = 1 \quad \forall i \in I$$

2) The maximum load capacity of a ship is never exceeded

$$\sum_{i \in I} q_i Y_{is} \leq Q_s \quad \forall s \in S$$

3) If ship s serves port j , it must have previously visited another port including the depot

$$\sum_{i \in I_0} X_{ijs} = Y_{js} \quad \forall j \in I \quad \forall s \in S$$

CONSTRAINTS

1) Each port is assigned to a ship

$$\sum_{s \in S} Y_{is} = 1 \quad \forall i \in I$$

2) The maximum load capacity of a ship is never exceeded

$$\sum_{i \in I} q_i Y_{is} \leq Q_s \quad \forall s \in S$$

3) If ship s serves port j , it must have previously visited another port including the depot

$$\sum_{i \in I_0} X_{ijs} = Y_{js} \quad \forall j \in I \quad \forall s \in S$$

CONSTRAINTS

1) Each port is assigned to a ship

$$\sum_{s \in S} Y_{is} = 1 \quad \forall i \in I$$

2) The maximum load capacity of a ship is never exceeded

$$\sum_{i \in I} q_i Y_{is} \leq Q_s \quad \forall s \in S$$

3) If ship s serves port j , it must have previously visited another port including the depot

$$\sum_{i \in I_0} X_{ijs} = Y_{js} \quad \forall j \in I \quad \forall s \in S$$

4) For each port and ship, the number of incoming arcs to j equals the number of outgoing arcs from j , ensuring flow conservation

$$\sum_{i \in I_0} X_{ijs} = \sum_{i \in I_0} X_{jis} \quad \forall j \in I \quad \forall s \in S$$

5) A ship may depart from the depot only if it serves at least one port

$$X_{0js} \leq \sum_{j \in I} Y_{js} \quad \forall s \in S$$

6) If a ship serves any port, it must depart from the depot

$$X_{0js} \geq \sum_{j \in I} \frac{Y_{js}}{I_{max}} \quad \forall s \in S$$

4) For each port and ship, the number of incoming arcs to j equals the number of outgoing arcs from j , ensuring flow conservation

$$\sum_{i \in I_0} X_{ijs} = \sum_{i \in I_0} X_{jis} \quad \forall j \in I \quad \forall s \in S$$

5) A ship may depart from the depot only if it serves at least one port

$$X_{0js} \leq \sum_{j \in I} Y_{js} \quad \forall s \in S$$

6) If a ship serves any port, it must depart from the depot

$$X_{0js} \geq \sum_{j \in I} \frac{Y_{js}}{I_{max}} \quad \forall s \in S$$

7) If ship s traverses arc (i, j) , then port j must appear after port i in the visit sequence

$$u_j \geq u_i + 1 - I_{max} \left(1 - \sum_{s \in S} X_{ijs}\right) \quad \forall i \in I \quad \forall j \in I \setminus i$$

8) If ship s travels from port i to port j , then the load at j must be at least the load at i minus the demand at i

$$l_{js} \geq l_{is} - q_i - Q_s (1 - X_{ijs}) \quad \forall i \in I \quad \forall j \in I \setminus i \quad \forall s \in S$$

9) The load of ship s upon entering port i must not exceed the port-specific loading limit L_{is}

$$l_{is} \leq L_{is} \quad \forall i \in I \quad \forall s \in S$$

10) The initial load of ship s equals the total demand of the ports it serves

$$l_{0s} = \sum_{i \in I} q_i Y_{is} \quad \forall s \in S$$

7) If ship s traverses arc (i, j) , then port j must appear after port i in the visit sequence

$$u_j \geq u_i + 1 - I_{max}(1 - \sum_{s \in S} X_{ijs}) \quad \forall i \in I \quad \forall j \in I0$$

8) If ship s travels from port i to port j , then the load at j must be at least the load at i minus the demand at i

$$l_{js} \geq l_{is} - q_i - Q_s(1 - X_{ijs}) \quad \forall i \in I \quad \forall j \in I0 \quad \forall s \in S$$

9) The load of ship s upon entering port i must not exceed the port-specific loading limit L_{is}

$$l_{is} \leq L_{is} \quad \forall i \in I \quad \forall s \in S$$

10) The initial load of ship s equals the total demand of the ports it serves

$$l_{0s} = \sum_{i \in I} q_i Y_{is} \quad \forall s \in S$$

7) If ship s traverses arc (i, j) , then port j must appear after port i in the visit sequence

$$u_j \geq u_i + 1 - I_{max} \left(1 - \sum_{s \in S} X_{ijs}\right) \quad \forall i \in I \quad \forall j \in I0$$

8) If ship s travels from port i to port j , then the load at j must be at least the load at i minus the demand at i

$$l_{js} \geq l_{is} - q_i - Q_s (1 - X_{ijs}) \quad \forall i \in I \quad \forall j \in I0 \quad \forall s \in S$$

9) The load of ship s upon entering port i must not exceed the port-specific loading limit L_{is}

$$l_{is} \leq L_{is} \quad \forall i \in I \quad \forall s \in S$$

10) The initial load of ship s equals the total demand of the ports it serves

$$l_{0s} = \sum_{i \in I} q_i Y_{is} \quad \forall s \in S$$

7) If ship s traverses arc (i, j) , then port j must appear after port i in the visit sequence

$$u_j \geq u_i + 1 - I_{max} \left(1 - \sum_{s \in S} X_{ijs}\right) \quad \forall i \in I \quad \forall j \in I0$$

8) If ship s travels from port i to port j , then the load at j must be at least the load at i minus the demand at i

$$l_{js} \geq l_{is} - q_i - Q_s (1 - X_{ijs}) \quad \forall i \in I \quad \forall j \in I0 \quad \forall s \in S$$

9) The load of ship s upon entering port i must not exceed the port-specific loading limit L_{is}

$$l_{is} \leq L_{is} \quad \forall i \in I \quad \forall s \in S$$

10) The initial load of ship s equals the total demand of the ports it serves

$$l_{0s} = \sum_{i \in I} q_i Y_{is} \quad \forall s \in S$$

VALID INEQUALITIES

These inequalities help the solver reduce solution time by eliminating infeasible or suboptimal routes early.

VI1) For each port j , if the total load of the ship s , to which it has been assigned, is greater than the maximum allowed load for s to enter j , then j cannot be the first port visited in the route

$$X_{0js} \leq 1 - \frac{1}{TOT_q} \left(\sum_{i \in I} q_i Y_{is} - L_{js} \right) \quad \forall j \in I \quad \forall s \in S$$

VI2) For each ship s and each pair of ports i and j , if the sum of their demand, q_i and q_j , is greater than the maximum allowed load for s to enter i , then j cannot be served immediately after i by ship s

$$X_{ijs} = 0 \quad \forall i \in I \quad \forall j \in J \quad \forall s \in S \mid q_i + q_j > L_{is}$$

VALID INEQUALITIES

These inequalities help the solver reduce solution time by eliminating infeasible or suboptimal routes early.

VI1) For each port j , if the total load of the ship s , to which it has been assigned, is greater than the maximum allowed load for s to enter j , then j cannot be the first port visited in the route

$$X_{0js} \leq 1 - \frac{1}{TOT_q} \left(\sum_{i \in I} q_i Y_{is} - L_{js} \right) \quad \forall j \in I \quad \forall s \in S$$

VI2) For each ship s and each pair of ports i and j , if the sum of their demand, q_i and q_j , is greater than the maximum allowed load for s to enter i , then j cannot be served immediately after i by ship s

$$X_{ijs} = 0 \quad \forall i \in I \quad \forall j \in J \quad \forall s \in S \mid q_i + q_j > L_{is}$$

VI3) Identify the earliest position a port i can occupy in the visiting sequence, without violating draft limit constraints, given the ship s to which it has been assigned and the set of ports assigned to it

$$p_s - (u_i - 1)q_{big} \leq L_{is} + Q_s(1 - Y_{is}) \quad \forall i \in I \quad \forall s \in S$$

$$q_{big} = \max_{i \in I} q_i$$

VI4) The latest position a port can assume in the visiting sequence is equal to the maximum number of ports that can be assigned simultaneously to the same ship, I^* .

I^* is computed by sorting ports in non-decreasing order of demand and counting how many can be assigned to the largest ship before exceeding its capacity.

$$u_i \leq I^* \quad \forall i \in I$$

VI3) Identify the earliest position a port i can occupy in the visiting sequence, without violating draft limit constraints, given the ship s to which it has been assigned and the set of ports assigned to it

$$p_s - (u_i - 1)q_{big} \leq L_{is} + Q_s(1 - Y_{is}) \quad \forall i \in I \quad \forall s \in S$$

$$q_{big} = \max_{i \in I} q_i$$

VI4) The latest position a port can assume in the visiting sequence is equal to the maximum number of ports that can be assigned simultaneously to the same ship, I^* .

I^* is computed by sorting ports in non-decreasing order of demand and counting how many can be assigned to the largest ship before exceeding its capacity.

$$u_i \leq I^* \quad \forall i \in I$$

Matheuristic

OVERVIEW

- The mathematical model efficiently solves only small instances
- For larger networks, the computational time grows exponentially
- To handle larger instances, two matheuristics are proposed:
 - **Large Neighborhood Search (LNS)**
 - **Iterated Local Search (ILS)**
- Both methods combine mathematical programming with local search principles

PARAMETERS TUNING

- Both the heuristics depend on two parameters
 1. m which is the number of nodes involved by the destroy operator
 2. α which is the proximity threshold
- The parameters were calibrated using the medium size instances
- The best resulting values are
 - $m = 5$
 - $\alpha = 1.5$

LARGE NEIGHBORHOOD SEARCH (LNS)

Main idea:

Use a randomized operator to partially destroy the solution and exploit the mathematical model to optimally rebuild a feasible solution starting from the partial solution obtained

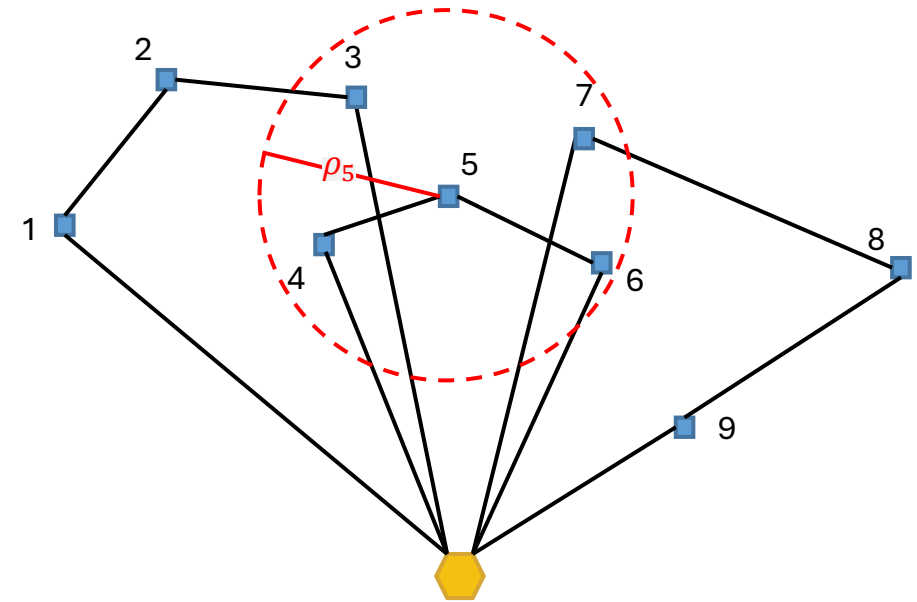
The destroy operator is

$$\rho_i = \alpha v_i$$

where v_i is the distance between the port i and its nearest port

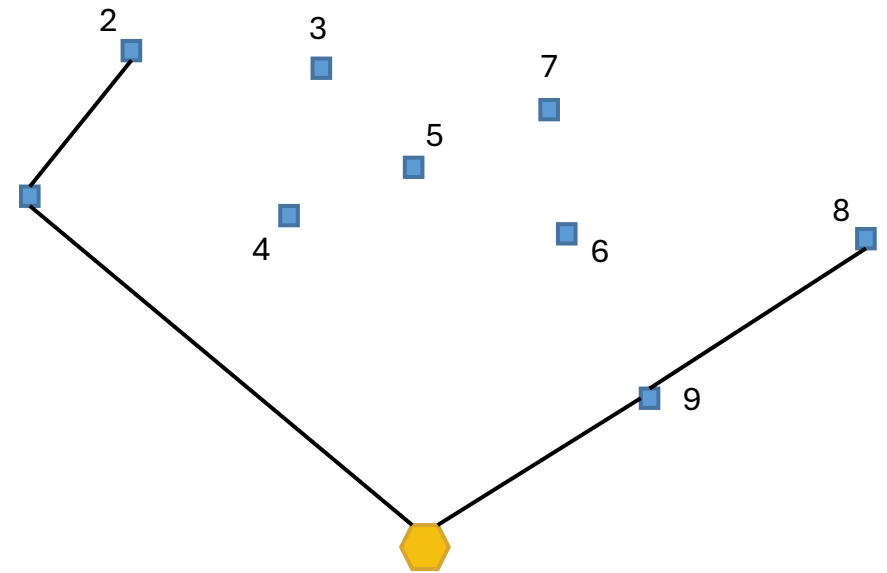
PROCEDURE

1. Run the mathematical model with a short time limit and keep the solution obtained so far
2. At each iteration are selected m random ports and removed from the solution
3. The destroy operator removes all the ports within a certain radius
4. The partially destroyed solution is passed to the model
5. If the obtained solution is better than the actual best solution, it is kept as current best, otherwise is discarded
6. The procedure is repeated until the stopping condition is reached



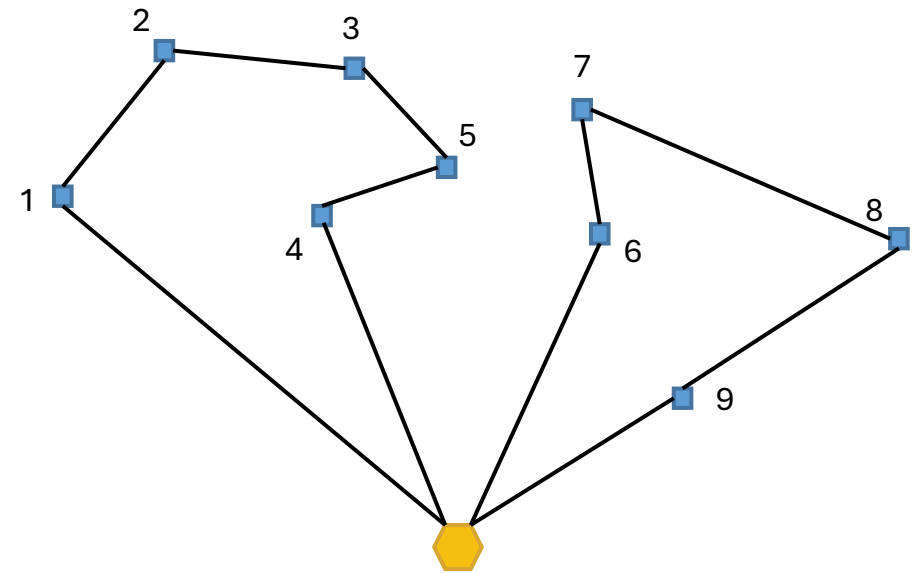
PROCEDURE

1. Run the mathematical model with a short time limit and keep the solution obtained so far
2. At each iteration are selected m random ports and removed from the solution
3. The destroy operator removes all the ports within a certain radius
4. The partially destroyed solution is passed to the model
5. If the obtained solution is better than the actual best solution, it is kept as current best, otherwise is discarded
6. The procedure is repeated until the stopping condition is reached



PROCEDURE

1. Run the mathematical model with a short time limit and keep the solution obtained so far
2. At each iteration are selected m random ports and removed from the solution
3. The destroy operator removes all the ports within a certain radius
4. The partially destroyed solution is passed to the model
5. If the obtained solution is better than the actual best solution, it is kept as current best, otherwise is discarded
6. The procedure is repeated until the stopping condition is reached





ITERATED LOCAL SEARCH (ILS)

Main idea:

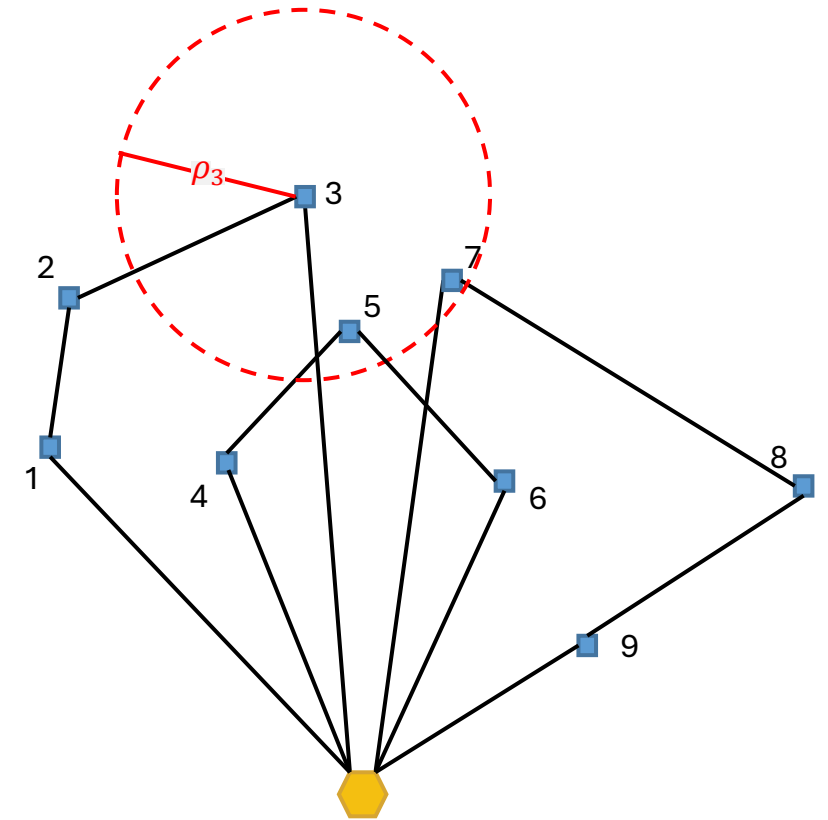
Combine a deterministic **Local Search (LS)** phase with a randomized **Diversification (DIV)** phase.

Algorithm outline:

1. Run the mathematical model with a short time limit and keep the solution obtained so far
2. Run the Local Search phase with the current solution
3. When the LS is trapped into a local minima Diversification phase is applied
4. LS and DIV are repeated until the stopping condition is reached

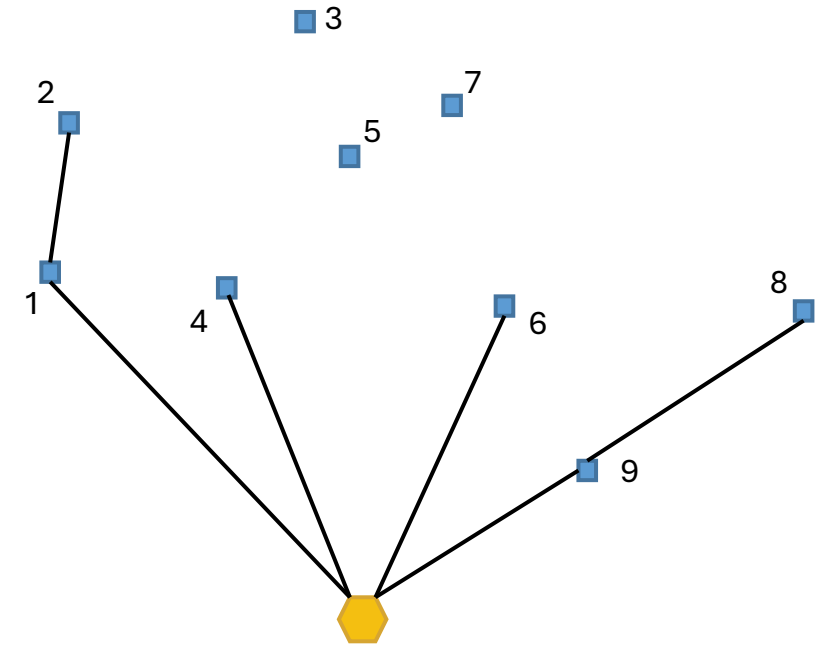
LOCAL SEARCH PROCEDURE

1. Every port is given a score $\sigma_i = \frac{\lambda_i}{v_i}$ where λ_i is the longest between the arc entering and the arc exiting the port i
2. The m ports with the highest score are removed together with their neighborhood within a certain radius ρ_i
3. Use the mathematical model to optimally rebuild the partial destroyed one
4. Accept the new solution if it improves the current best, otherwise executes DIV



LOCAL SEARCH PROCEDURE

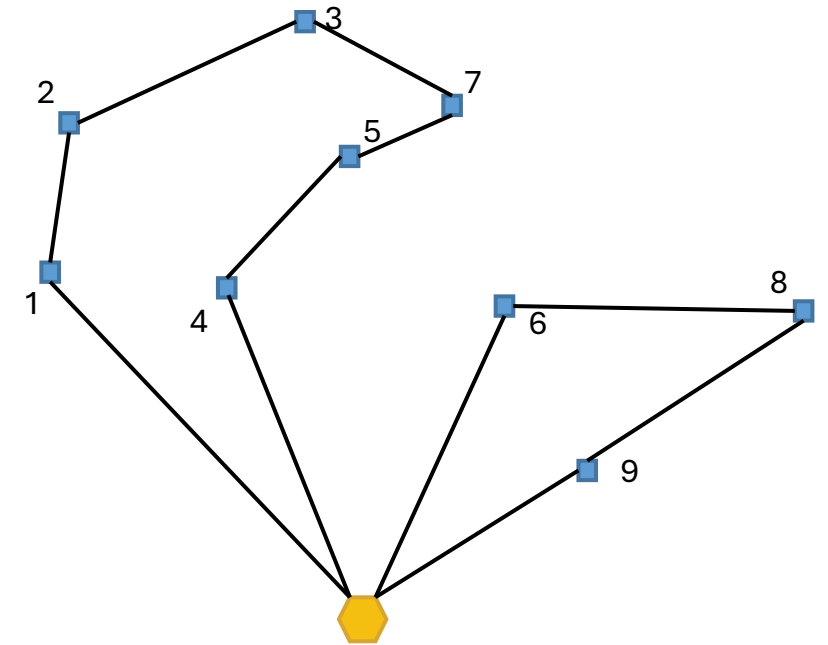
1. Every port is given a score $\sigma_i = \frac{\lambda_i}{v_i}$ where λ_i is the longest between the arc entering and the arc exiting the port i
2. The m ports with the highest score are removed together with their neighborhood within a certain radius ρ_i
3. Use the mathematical model to optimally rebuild the partial destroyed one
4. Accept the new solution if it improves the current best, otherwise executes DIV





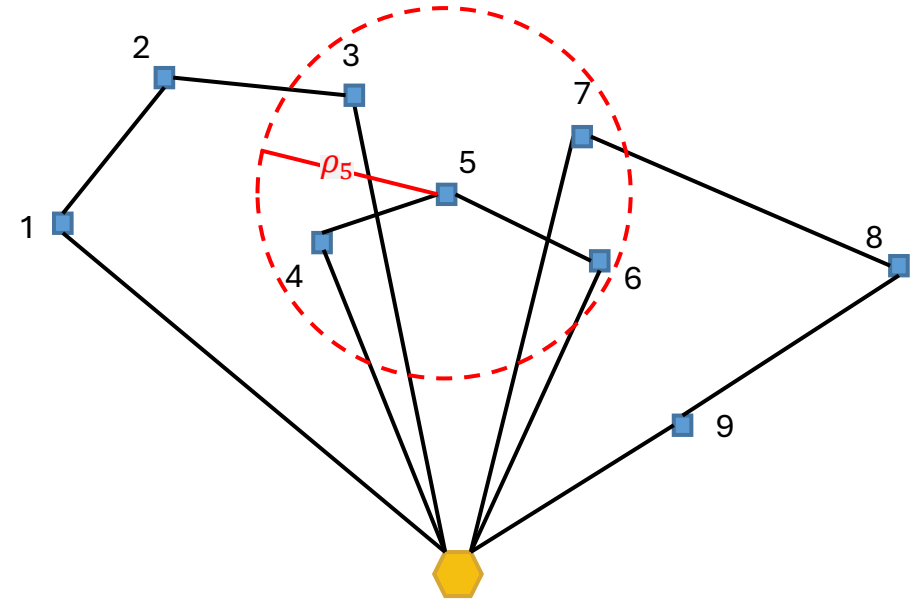
LOCAL SEARCH PROCEDURE

1. Every port is given a score $\sigma_i = \frac{\lambda_i}{v_i}$ where λ_i is the longest between the arc entering and the arc exiting the port i
2. The m ports with the highest score are removed together with their neighborhood within a certain radius ρ_i
3. Use the mathematical model to optimally rebuild the partial destroyed one
4. Accept the new solution if it improves the current best, otherwise executes DIV



DIVERSIFICATION PROCEDURE

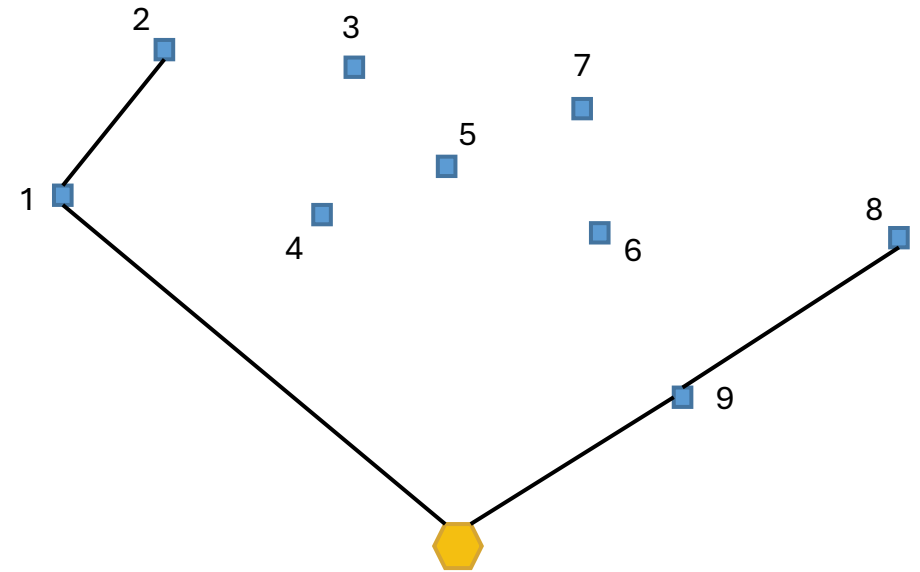
1. Randomly select m ports to remove from the best current solution
2. Include also nearby ports within a distance ρ_i
3. Use the mathematical model to optimally rebuild the partial destroyed one
4. The obtained solution, even if it is worse, is given as input to the LS





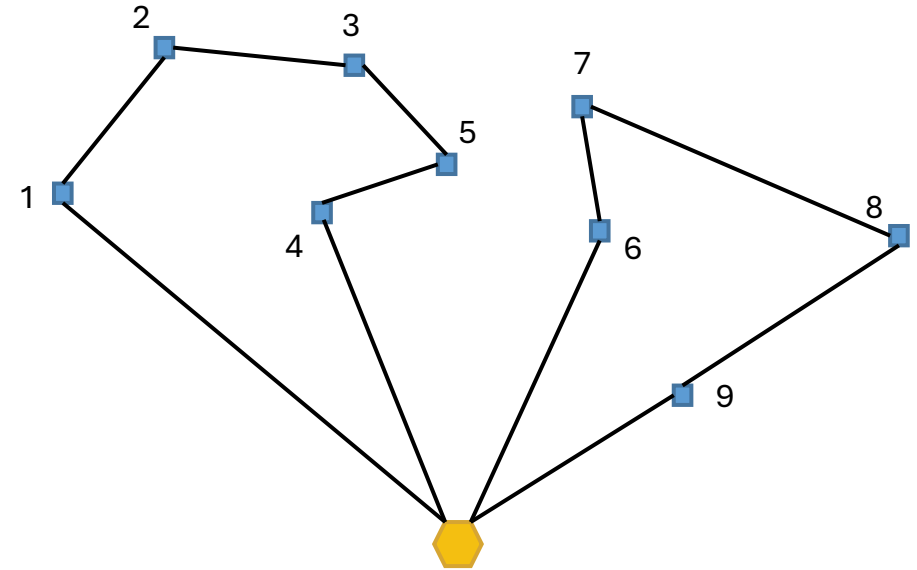
DIVERSIFICATION PROCEDURE

1. Randomly select m ports to remove from the best current solution
2. Include also nearby ports within a distance ρ_i
3. Use the mathematical model to optimally rebuild the partial destroyed one
4. The obtained solution, even if it is worse, is given as input to the LS



DIVERSIFICATION PROCEDURE

1. Randomly select m ports to remove from the best current solution
2. Include also nearby ports within a distance ρ_i
3. Use the mathematical model to optimally rebuild the partial destroyed one
4. The obtained solution, even if it is worse, is given as input to the LS



Dataset

- Parameters
 - Number of ships $s \in [3, 5, 6, 10]$
 - Number of ports $i \in [15, 25, 50]$
 - 2 scenarios for Draft Restriction (DR):
 - $LOW_{DR} = 30\%$ of ports affected by draft limits
 - $HIGH_{DR} = 70\%$ of ports affected by draft limits
 - 2 scenarios for Capacity Tightness ($CT = \frac{TOT_q}{TOT_Q}$)
 - $LOW_{CT} = 30\%$ of ship capacity corresponds to the total demand
 - $HIGH_{CT} = 70\%$ of ship capacity corresponds to the total demand

SMALL INSTANCES

- 40 small instances divided in 4 sets of 10 instances each
 - **Set 1:**
 - 15 ports
 - 3 ships
 - $(LOW_{DR} - LOW_{CT})$
 - **Set 2:**
 - 15 ports
 - 3 ships
 - $(HIGH_{DR} - LOW_{CT})$
 - **Set 3:**
 - 15 ports
 - 3 ships
 - $(LOW_{DR} - HIGH_{CT})$
 - **Set 4:**
 - 15 ports
 - 3 ships
 - $(HIGH_{DR} - HIGH_{CT})$

MEDIUM & LARGE INSTANCES

- 22 medium instances divided in 2 sets of 11 instances each
 - **Set 5:**
 - 25 ports
 - 5 ships
 - $(HIGH_{DR} - HIGH_{CT})$
 - **Set 6:**
 - 25 ports
 - 6 ships
 - $(HIGH_{DR} - HIGH_{CT})$
- 10 large instances in 1 set
 - **Set 7:**
 - 50 ports
 - 10 ships
 - $(HIGH_{DR} - HIGH_{CT})$

Results - mathematical model

- For each set, two tables are shown:
 - The first reports the average of objective function value, lower bound, and gap obtained.
 - The second reports the average computation time and the gap achieved for each of the configurations considered.
- For the medium and large sized sets (sets 5, 6, and 7), the mathematical model was configured with a time limit of 600 seconds.

Mathematical model – Set 1

AVG OF	AVG OBJ BOUND	AVG MIP GAP
307,063	307,063	OPT

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	16,757	14,233	17,611	13,907	9,526	9,447	9,367	21,613	21,942
AVG MIP GAP	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT

Mathematical model – Set 2

AVG OF	AVG OBJ BOUND	AVG MIP GAP
309,577	308,269	OPT

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	14,753	9,924	14,529	11,435	9,839	7,302	9,300	22,872	17,779
AVG MIP GAP	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT

Mathematical model – Set 3

AVG OF	AVG OBJ BOUND	AVG MIP GAP
442,359	439,630	OPT

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	57,709	43,258	56,489	128,282	150,548	57,017	28,059	54,539	48,892
AVG MIP GAP	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT



Mathematical model – Set 4

AVG OF	AVG OBJ BOUND	AVG MIP GAP
443,712	441,354	OPT

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	48,722	23,758	74,850	41,460	33,429	13,799	34,007	49,130	21,937
AVG MIP GAP	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT	OPT

Mathematical model – Set 5

AVG OF	AVG OBJ BOUND	AVG MIP GAP
750,348	604,516	0,194

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	601,222	601,297	601,299	601,194	601,421	601,697	601,441	601,281	601,301
AVG MIP GAP	0,183	0,182	0,182	0,189	0,192	0,197	0,189	0,223	0,206

Mathematical model – Set 6

AVG OF	AVG OBJ BOUND	AVG MIP GAP
697,740	538,662	0,228

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	600,423	600,376	600,432	600,360	657,861	639,458	633,259	638,205	640,450
AVG MIP GAP	0,230	0,221	0,225	0,229	0,229	0,233	0,227	0,234	0,228



Mathematical model – Set 7

AVG OF	AVG OBJ BOUND	AVG MIP GAP
905,186	552,166	0,387

	NO VI	VI1	VI2	VI3	VI4	VI1+VI4	VI2+VI4	VI3+VI4	ALL VI
AVG COMPUTATIONAL TIME (s)	602,642	602,995	603,113	602,779	602,540	683,787	602,556	616,780	602,489
AVG MIP GAP	0,352	0,415	0,373	0,369	0,357	0,423	0,351	0,402	0,439

Results - matheuristic

	LNS		ILS	
SET	AVG GAP TO MODEL	AVG TIME	AVG GAP TO MODEL	AVG TIME
1	0	20,754	0	23,063
2	0	16,386	0	18,991
3	0	31,604	0	35,950
4	0	28,875	0	45,278
5	-0,0345	172,255	-0,0325	125,760
6	-0,0144	137,114	-0,0610	286,325
7	-0,1635	245,695	-0,1811	339,918

The table shows the average heuristic gap with respect to the best upper bound found by the MILP (VI1+VI4), together with average runtime for LNS and ILS.

Matheuristic VS Mathematical Model

	MODEL VI1 + VI4		LNS		ILS	
	AVG GAP TO LB	AVG TIME	AVG GAP TO LB	AVG TIME	AVG GAP TO LB	AVG TIME
1	0	9,447	0	20,754	0	23,063
2	0	7,302	0	16,386	0	18,991
3	0	57,017	0	31,604	0	35,950
4	0	13,799	0	28,875	0	45,278
5	0,182	601,421	0,2112	172,255	0,2292	125,760
6	0,221	639,458	0,2968	137,114	0,2881	286,325
7	0,351	683,787	0,4515	245,695	0,4282	339,918

The table shows the average heuristic gap computed relative to the model's lower bound (best configuration VI1+VI4), along with average runtime for each method.

Conclusions

- Developed and implemented a **mathematical model** for the HF-VRP-DL, integrating heterogeneous fleet assignment and draft-based routing constraints.
- Introduced **valid inequalities** that reduced computation times for small instances and improved model tractability.
- Designed and tested two **matheuristic approaches** (LNS and ILS) that provided better solutions with lower computational effort on medium and large instances.